

SOFTWARE REQUIREMENTS SPECIFICATION

Version 1.1

Marketplace Order Orchestrator

RDBMS & SOLID Architecture Edition

Prepared by: Mai Ahmed Abdellatif

Document Reference	MOO-SRS-2025-001
Project Title	Marketplace
Document Version	1.1
Status	APPROVED — BASELINE
Prepared For	Academic Software Engineering Project
Database Platform	MySQL 8.x (via JDBC)
Primary Language	Java 17+
Date	2026-06-22

TTABLE OF CONTENTS

1. Project Overview & Scope.....2

 1.1 Purpose.....2

 1.2 Scope.....5

 1.3 Definitions, Acronyms, and Abbreviations.....6

 1.4 References.....7

 1.5 Overview.....7

2. Overall Description.....8

 2.1 Product Perspective & Architecture.....8

 2.2 Product Functions.....9

 2.3 User Characteristics.....10

 2.4 Constraints.....11

 2.5 Assumptions and Dependencies.....12

 2.6 Apportioning of Requirements.....12

3. Specific Requirements.....13

 3.1 External Interface Requirements.....13

 3.2 Functional Requirements.....15

 3.2.1 Catalog & Inventory Management.....15

3.2.2 Order Lifecycle & State Machine.....16

3.2.3 Promotions & Shipping Engine.....16

3.2.4 Reporting Matrix.....17

3.3 Performance Requirements.....17

3.4 Logical Database Requirements.....18

3.5 SOLID & OOP Architectural Constraints.....19

3.6 Software System Attributes (NFRs).....20

4. Supporting Information.....21

4.1 Table of Contents.....21

4.2 Index.....21

4.3 Appendixes.....21

Revision History

Version	Date	Description	Author
1.0	2026-06-09	Initial baseline release — covers full domain model, functional requirements, DB schema, SOLID constraints, and reporting matrix.	Project Team
1.1	2026-06-22	Updated SRS template to align with standard guidelines, and clarified validation and business rules within functional requirements.	Project Team

1. Project Overview & Scope

1.1 Purpose

This document defines the complete functional and non-functional requirements for the Marketplace Order Orchestrator System — a high-integrity backend orchestration engine. The system manages multi-vendor product catalogs, end-to-end order lifecycles, inventory allocation, promotional discount engines, shipment routing, and auditable analytics reporting.

This SRS serves as the contractual baseline between all project stakeholders and the engineering team. All implementation decisions must be traceable to at least one requirement stated herein.

1.2 Scope

This document applies to the **Marketplace Order Orchestrator System**.

1.2.1 In-Scope (System Capabilities) The system is responsible for the following core backend operations:

- Multi-vendor product catalog with centralized inventory management.
- Transactional order lifecycle state machine (CREATED → ALLOCATED → PACKED → SHIPPED).
- Stock allocation engine supporting full, partial, and zero-stock scenarios.
- Pluggable promotional discount framework (e.g., Percentage, Threshold-Fixed, SKU Bundle).
- Carrier routing engine with Round-Robin load balancing via configurable rules.
- Auditable event ledger capturing every state transition and administrative action.
- Structured analytics and performance reporting matrix.
- Intelligent returns processing workflow with automatic inventory restocking and order state reconciliation.

1.2.2 Out-of-Scope

To clearly define the system boundaries, the following are explicitly excluded from this system:

- The system will NOT render User Interfaces (UI) for end-users or sellers (it exposes RESTful APIs for consumption).
- The system will NOT process direct credit card payments or store raw financial data (it relies on integrations with external payment gateways).
- The system will NOT manage the physical logistics, tracking hardware, or fleet routing of delivery vehicles.

1.2.3 Objectives and Benefits

The primary objective of this system is to provide a highly scalable, open marketplace backend. It is designed to orchestrate complex multi-vendor orders seamlessly, completely eliminate overselling through strict inventory

allocation rules, and deliver an auditable, high-performance infrastructure capable of growing with increasing transaction volumes.

1.3 Definitions, Acronyms, and Abbreviations

This subsection provides the definitions of all terms, acronyms, and abbreviations required to properly interpret this SRS:

- **API** (Application Programming Interface): A set of rules and protocols that allows different software applications to communicate with each other.
- **Compliance**: Adherence to legal, regulatory, industry, or technical standards and policies (e.g., data privacy laws, payment processing standards, or internal auditing rules) required for the system's secure and lawful operation.
- **Ledger**: An append-only, immutable database table or log used to record a history of transactions and state changes for auditing purposes.
- **Orchestrator**: The central backend engine responsible for coordinating multiple services (e.g., inventory, ordering, routing) to complete a complex business workflow.
- **Round-Robin**: A load-balancing algorithm that distributes tasks (e.g., assigning orders to shipping carriers) sequentially and equally among all available options.
- **SKU** (Stock Keeping Unit): A unique alphanumeric identifier used to track a specific product and its variants in inventory.
- **State Machine**: A mathematical model of computation used to manage the lifecycle of an order, ensuring it moves only through predefined valid states (e.g., from CREATED to ALLOCATED).
- **Zero-Stock Scenario**: A condition where a product's available quantity reaches zero, triggering specific business rules (e.g., hiding the product or marking it as 'Out of Stock').
- **Allocation**: The act of reserving available product stock against a requested order line quantity.
- **Audit Event**: An immutable record of a system action, including actor identity, action type, timestamp, and optional details.
- **ORM** (Object-Relational Mapper): A programming technique (e.g., Hibernate, JPA) used to convert data between incompatible type systems. (*Note: Explicitly prohibited in this project as per design constraints*).
- **JDBC** (Java Database Connectivity): The standard Java API for relational database access.

1.4 References

1. Title: IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).

- **Date:** June 1998.
 - **Publishing Organization:** IEEE Computer Society.
 - **Source:** IEEE Xplore Digital Library.
2. **Title:** Logistics & Carrier API Integration Guide, v2.1.
- **Date:** January 2026.
 - **Publishing Organization:** FastRoute Shipping Inc.
 - **Source:** Available on the internal engineering drive at <https://www.bing.com/ck/a?!&&p=19b3b5d9a95315a3cb63e9f27a4e46a9ca999e53736dd88ce52d4dcc3eb15f69Jm1tdHM9MTc4MjAwMDAwMA&ptn=3&ver=2&hsh=4&fclid=3c503710-14cc-6c13-2746-224115b06d54&psq=IEEE+830+Software+Requirements+Specification+example+filetype%3apdf&u=a1aHR0cDovL3d3dy5tYXRoLnVhYS5hbGFza2EuZWRR1L35hZmtqbS9jczQwMS9JRUVFODMwLnBkZg>

1.5 Overview

The remainder of this Software Requirements Specification (SRS) is organized to provide clear guidance and easy navigation for all stakeholders. The following table outlines the structure and contents of the document:

Section / Component	Target Audience	Description and Contents
Section 2: Overall Description	All Stakeholders (Project Managers, Clients, Developers)	Provides a high-level background of the system. It covers the architectural perspective, user roles (Buyers, Sellers, Admins), operating environment, and general business constraints.
Section 3: Specific Requirements	Engineering & QA Teams (Developers, Testers, Architects)	Contains the exhaustive, technical, and testable requirements. It details external API interfaces, functional logic (e.g., Stock Allocation, Order State Machine), database

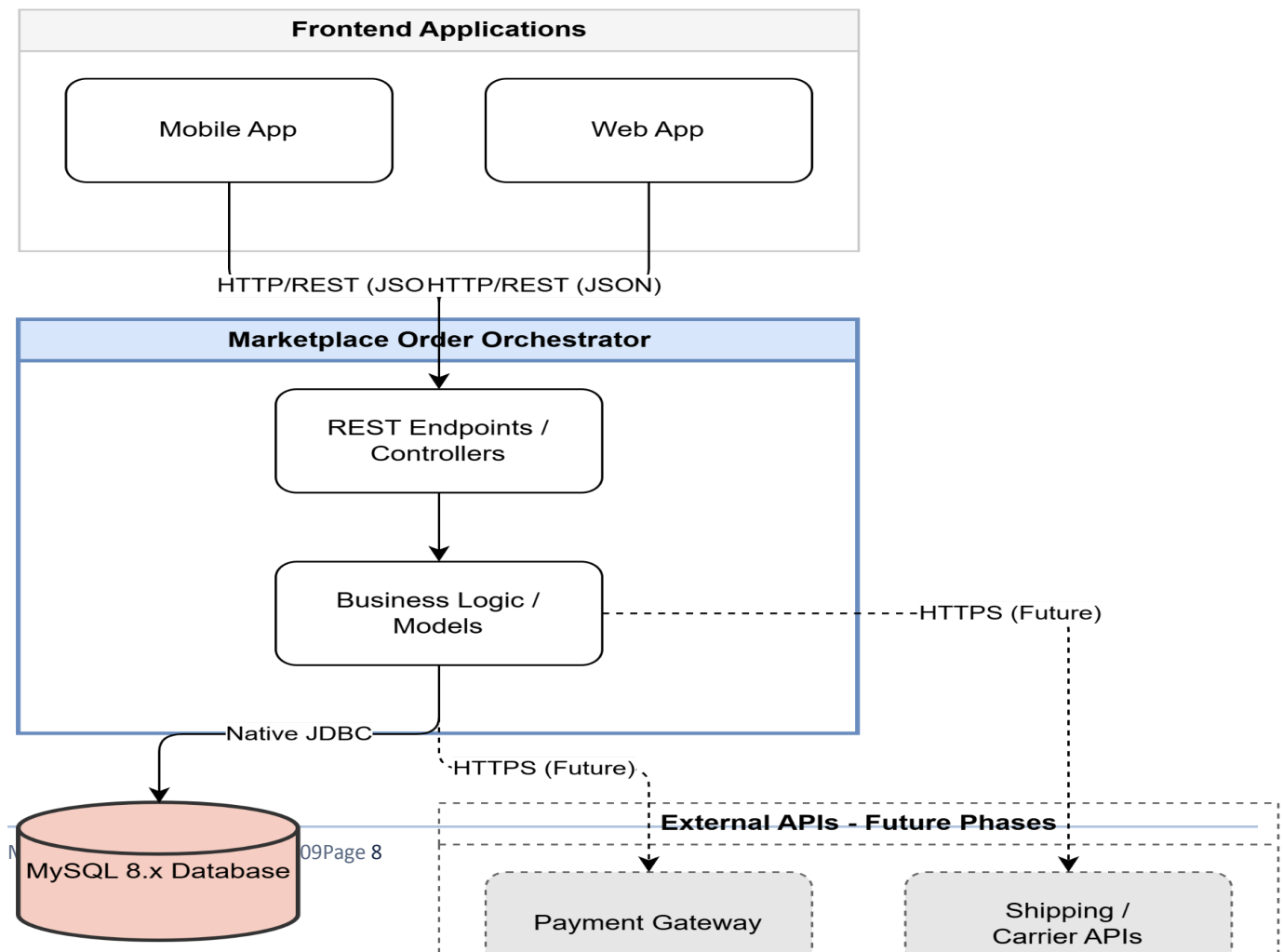
		constraints (e.g., MySQL 8.x, JDBC), and performance targets.
Supporting Information	General Readers & Implementers	Contains the document index and any appendices used to supplement the specification (e.g., external API JSON payloads or reference materials).

2.1 Product Perspective & Architecture

The Marketplace Order Orchestrator operates as a standalone backend subsystem within a broader e-commerce ecosystem. Internally, the system architecture strictly adheres to the MVC (Model-View-Controller) design pattern tailored for RESTful API services:

- **Controllers:** Act as the entry points, exposing stateless HTTP REST endpoints to external Web and Mobile frontends.
- **Models:** Contain the core business domain logic (e.g., Order State Machine, Inventory Allocation) and interact securely with the internal MySQL 8.x database via native JDBC.
- **Views:** Represent the serialized JSON payloads (Data Transfer Objects) returned to the external clients, as the system does not render graphical user interfaces.

System Context Diagram:



2.2 Product Functions

The Marketplace Order Orchestrator provides the following primary functions to manage the end-to-end order lifecycle:

- **Centralized Order Orchestration:**
Manages the complete lifecycle of multi-vendor orders, ensuring consistent state transitions from creation through allocation to fulfillment.
- **Intelligent Inventory Management:**
Handles automated stock allocation and real-time availability tracking across multiple seller catalogs, effectively preventing overselling.
- **Dynamic Promotional Framework:**
Executes complex discount calculations (e.g., percentage, fixed-threshold, and bundle discounts) seamlessly during the order processing phase.
- **Carrier Routing & Logistics:**
Automates the assignment of shipping carriers to orders based on predefined business rules and load-balancing algorithms (Round-Robin).
- **Automated Returns Processing:**
Manages the reverse logistics workflow, including the automatic reconciliation of order states and the seamless restocking of returned inventory.
- **Audit & Performance Reporting:**
Maintains an immutable ledger of all system events and provides structured analytical reports to monitor system performance and seller activity.
- **User & Role Management:**
Manages user identity and role assignment, strictly enforcing a one-account-per-user policy. It provides administrative controls to register users, verify credentials, and update user roles (e.g., upgrading a regular user to a seller or vice versa) while maintaining data integrity across the platform.

2.3 User Characteristics

This section describes the profile of the system's end-users. The system is designed to accommodate users with varying levels of technical expertise.

User Role	Description & Background	Technical Proficiency
System Admin	Manages the platform, monitors system performance, and handles user role transitions (e.g., promoting/demoting accounts).	High (Technical)
Seller	Manages product catalogs, stock levels, and monitors order fulfillment. They expect a reliable interface to update availability and track sales.	Medium
Customer	Initiates orders, browses products, and manages returns. They expect a seamless, fast, and highly available shopping experience.	Low (Non-Technical)

2.4 Constraints

This section specifies the constraints that limit the developer's design options and mandate specific standards to be followed:

- **Regulatory Policies:** The system must comply with data privacy and security standards regarding the handling of customer PII (Personally Identifiable Information).
- **Hardware Limitations:** The system must be capable of running on standard cloud-based virtual machine instances (e.g., 2 vCPU, 4GB RAM) without resource exhaustion.

- **Interfaces to Other Applications:** The system must interface with external REST-based Payment Gateways and Carrier APIs, utilizing standard HTTPS/JSON communication protocols.
- **Parallel Operation:** The system must be capable of handling concurrent requests from multiple frontend instances (Web and Mobile) without race conditions during inventory allocation.
- **Audit and Control Functions:** An immutable audit trail of all order status transitions must be maintained, as required for financial and operational compliance.
- **Design and Implementation Standards(Technical):**
 - Language & Runtime: Java 17 LTS or higher.
 - Persistence: Native JDBC with MySQL 8.x; ORM frameworks are strictly prohibited.
 - Coding Standards: Strict adherence to SOLID principles, with **MAJOR** severity for violations during code review.
 - Build/Tooling: Standardized Maven or Gradle lifecycle management.

2.5 Assumptions and Dependencies This section outlines the factors that, if changed, would affect the system requirements:

- **Assumptions:**
 - Connectivity: It is assumed that the application server has stable, low-latency access to the MySQL database and the internet for external API calls.
 - Data Accuracy: It is assumed that the pricing and inventory data provided by sellers via the API/Interface are accurate at the time of entry.
 - Operational Environment: It is assumed that the runtime environment supports Java 17+ and standard shell execution for the CLI layers.
- **Dependencies:**
 - External APIs: The system is dependent on the availability and correct response structure of external Payment Gateways and Carrier APIs (e.g., FastRoute Shipping).
 - Infrastructure: The system depends on a standard RDBMS environment (MySQL 8.x) for all data persistence and transaction management.

2.6 Apportioning of Requirements This subsection identifies requirements that are delayed for future releases:

- **Phase 1 (Current Release):** Covers core orchestration, inventory allocation, state machine enforcement, and internal reporting.
 - **Phase 2 (Future Release):** Will include the full integration of real-time third-party Payment Gateway APIs and automated Carrier shipping label generation.
-

3. Specific Requirements

This section details the functional and technical requirements of the Marketplace Order Orchestrator. It specifies the necessary interfaces and data structures required for the system to perform its core operations.

3.1 External Interface Requirements

As a backend orchestration engine, the system does not provide a graphical user interface (GUI). Instead, it exposes a set of RESTful APIs that allow external applications (such as Web or Mobile frontends) to interact with the system.

- **Communication Protocol:** All system interactions must occur over secure HTTPS.
- **Data Format:** The system exclusively uses JSON (JavaScript Object Notation) for all data exchanges, ensuring a lightweight and standardized format for requests and responses.
- **Interaction Model:** External applications act as clients that send requests (API calls) to the Orchestrator. The system processes these requests based on business rules and returns a structured response indicating success or failure.
- **Integration Requirements:** To ensure system security and reliability, all API calls must be authenticated, and the system is designed to handle multiple concurrent

requests without data corruption, strictly adhering to the database constraints defined in Section 3.

3.1.1 External Interface Requirements

3.1.1.1 User Interfaces

- **Logical Characteristics:** Since the system is an API-first backend, there is no graphical user interface (GUI). The "interface" consists of structured JSON data objects. All reports (RPT-001 to RPT-006) are generated as formatted text streams (via **StringBuilder**) designed for human-readable output in a console or log viewer.
- **Optimization:** The system provides two levels of error messaging: a standard **SHORT** message for automated systems and a **VERBOSE** message for administrative debugging. All API response structures must be consistent; a first-time developer should be able to integrate an endpoint (e.g., POST **/orders**) within 30 minutes of consulting this documentation.

3.1.1.2 Hardware Interfaces

- The system is hardware-agnostic and runs on standard x86_64 server architectures.
- It requires network interface support (NIC) for TCP/IP communication (HTTPS/JSON).
- No specific peripheral or physical hardware control (e.g., scanners, printers) is required by this software product.

3.1.1.3 Software Interfaces

- **Operating System:** Linux (Ubuntu 22.04 LTS or higher) or Windows Server 2022.
- **Database Management System:** MySQL 8.0.x (Source: Oracle Corp).
- **Runtime Environment:** Java SE Development Kit (JDK) 17 LTS (Source: Eclipse Adoptium/OpenJDK).
- **Integration:** The system interfaces with external Payment Gateways and Carrier APIs (FastRoute Shipping, v2.1). Connectivity is established via standard HTTPS calls, requiring the external services to support JSON-based request/response payloads.

3.1.1.4 Communications Interfaces

- The system communicates over local area networks (LAN) and wide area networks (WAN) using the TCP/IP protocol suite.
- All data transfer is secured via TLS 1.2+ encryption.
- No proprietary communication protocols are used.

3.1.1.5 Memory Constraints

- **Primary Memory (RAM):** The application heap size is capped at 4GB. The system must operate efficiently within this limit. Use of `ArrayList` is permitted, but memory-intensive operations (like large report generation) must be buffered to avoid `OutOfMemoryError`.
- **Secondary Memory (Disk):** Minimum 10GB of available disk space for initial database records and application logs. Storage requirements will scale linearly with transaction volume.

3.1.1.6 Operations

- **Modes of Operation:**
 - **User-Initiated:** All product management, order placement, and reporting are triggered by authenticated API requests.
 - **Unattended:** The system performs automated background tasks, such as periodic log cleanup and scheduled audit consistency checks.
- **Data Processing:** Support functions include automated inventory reconciliation and carrier assignment during the allocation lifecycle.
- **Backup & Recovery:** Database integrity is maintained via binary logging. Full database backups are performed daily at 00:00 UTC.

3.1.1.7 Site Adaptation Requirements

- **Initialization:** All environment-specific variables (DB Credentials, API Keys, Base URLs, Reorder Thresholds) must be provided via an external `application.properties` or `config.json` file at runtime.
- **Installation Adaptation:** The system is designed to be environment-agnostic. No hardcoded site-specific paths or local configurations are permitted; all adaptations to the local environment must be facilitated through the configuration file during the deployment phase.

3.2 Functions

The system performs the following core operations to orchestrate the order lifecycle. All operations strictly adhere to atomic persistence and transactional integrity.

3.2.1. Functional Requirements

All functional requirements are stated as mandatory system behaviors.

001 Catalog & Inventory Management

- **FR-CAT-001 (Product Ingestion):** The system shall accept new product records in batches (max 100/request). The system shall validate SKU (numeric), Title (1-100 chars), SellerID (existing), AvailableQty (≥ 0), and Price (> 0). If any validation fails, the system shall perform a full rollback of the batch and return an error. On success, the system shall persist data and generate a `PRODUCT_CREATED` audit record.
- **FR-CAT-002 (Product Visibility Toggle):** The system shall allow administrators to toggle the `active` status of any product. When `active` is set to `FALSE`, the system shall reject all new order lines containing that SKU. All toggles shall be recorded in the audit log.
- **FR-CAT-003 (Audited Stock Adjustment):** The system shall allow administrators to manually modify `availableQty` for any product. The system shall require a non-empty justification string for every adjustment. The system shall reject any adjustment that results in a negative `availableQty`. The system shall persist the justification in the `AuditEvent` ledger.

002 Order Lifecycle & State Machine

- **FR-ORD-000 (State Machine Enforcement):** The system shall strictly enforce the linear order state machine (`CREATED` $\rightarrow$ `ALLOCATED` $\rightarrow$ `PACKED` $\rightarrow$ `SHIPPED`). The system shall reject any transition that violates the predefined state sequence.
- **FR-ORD-001 (Stock Allocation Engine):** The system shall support three allocation modes:
 - **Full:** If `AvailableQty` $\geq$ `Requested`, the system shall deduct the full quantity.
 - **Partial:** If $0 < \text{AvailableQty} < \text{Requested}$, the system shall deduct all remaining stock and flag the unmet quantity.
 - **Zero:** If `AvailableQty` $= 0$, the system shall reject allocation and inform the customer.
- **FR-ORD-002 (Order Cancellation):** The system shall allow cancellation only in `CREATED`, `ALLOCATED`, or `PACKED` states. The system shall atomically restore all previously deducted inventory to the respective products and emit an `ORDER_CANCELLED` audit event.
- **FR-ORD-003 (Refund Processing):** The system shall allow refund processing only for `SHIPPED` orders. The system shall transition the status to `REFUNDED` without restoring inventory and emit an `ORDER_REFUNDED` audit event.

003 Promotions & Shipping Engine

- **FR-PROMO-001 (Promotion Application):** The system shall calculate discounts sequentially according to the configured promotion order. The system shall ensure the grand total is never negative by clamping the effective discount to the subtotal. The system shall include all applied promotion details in the order receipt.

- **FR-SHIP-001 (Carrier Configuration):** The system shall load shipping carriers, standard fees, and free-shipping thresholds from an external JSON configuration file at startup.
- **FR-SHIP-002 (Round-Robin Assignment):** The system shall assign shipping carriers using a stateful Round-Robin algorithm to ensure sequential and equitable distribution.
- **FR-SHIP-003 (Tracking Generation):** The system shall generate a unique, monotonically increasing tracking number for every shipment using an atomic counter.

004 Reporting Matrix

- **FR-RPT-001 (Order Summary):** The system shall generate a financial snapshot of all orders, filterable by status, customer, or date.
- **FR-RPT-002 (Inventory Status):** The system shall generate a real-time stock snapshot, grouped by seller, highlighting current inventory value.
- **FR-RPT-003 (Revenue by Seller):** The system shall calculate gross and net revenue per seller, excluding CANCELLED and REFUNDED orders.
- **FR-RPT-004 (Shipment Tracking):** The system shall generate an operational view of shipments, grouped by carrier, to monitor delivery status.
- **FR-RPT-005 (Audit Trail):** The system shall provide a chronological ledger of all system actions, filterable by actor or event type.
- **FR-RPT-006 (Low-Stock Alert):** The system shall identify and report products where `availableQty` falls below a user-defined threshold.
- **FR-RPT-000 (Global Trigger):** System Admins shall be authorized to trigger any of the above reports through the administrative interface.

3.3 Performance Requirements

This subsection defines the quantitative performance metrics that the system must satisfy under both normal and peak operational conditions.

1. Static Numerical Requirements (Capacity)

- **Simultaneous Users:** The system shall support up to 50 concurrent administrative and seller sessions without degradation in response time.
 - **Product Catalog Capacity:** The system shall support a catalog size of up to 100,000 active SKU records stored in the RDBMS.
 - **Information Throughput:** The system shall be capable of handling an aggregate data volume of 500MB per day, including audit logs and transaction history.
-

2. Dynamic Numerical Requirements (Throughput & Latency)

- **Transaction Latency (Normal Load):** 95% of all API transactions (e.g., placing an order, stock adjustment) shall be processed in less than 200ms.
 - **Transaction Latency (Peak Load):** Under a peak load of 50 transactions per second (TPS), 95% of all transactions shall be processed in less than 500ms.
 - **Report Generation:** The system shall generate any of the six defined analytical reports (Section 4.4) for a dataset of 10,000 records in less than 3 seconds.
 - **Audit Logging:** The system shall persist **AuditEvent** records to the database within 50ms of the triggered action.
 - **Report Generation:** The system shall generate the requested analytical reports for the daily transaction volume (100 orders) in less than 1 second, ensuring minimal impact on ongoing order processing.
-

3.4 Logical Database Requirements

The backend engine utilizes a structured relational persistence layer to guarantee strict transactional integrity and facilitate high-performance complex querying across the domain model.

- **Information Domain Architecture:** The RDBMS manages the following core entities:
 - **Identity & Party Schema:** Centralized profiles for Seller and Customer accounts.
 - **Product Catalog Ledger:** Granular SKU metadata, financial pricing models, and real-time stock levels.
 - **Order Lifecycle State:** Transactional headers, polymorphic line items, and historical state machine transitions.
 - **Logistics & Routing:** Carrier routing assignments and monotonically increasing tracking identifiers.
 - **Auditable Event Store:** Immutable logs capturing every administrative toggle and state reconciliation.
- **Access & Utilization Patterns:** Operational records (Orders, Inventory) sustain continuous real-time modifications, while analytical reports are derived via on-demand aggregate queries by authorized personnel.
- **Persistence Layer Mechanisms:** All data interactions must leverage native **JDBC** execution to ensure full ACID compliance without the overhead of prohibited abstraction layers.
- **Relational Integrity Constraints:** The schema enforces highly normalized relationships through mandatory Foreign Key constraints to safeguard referential consistency across the platform.
- **Database Integrity Enforcement:** Critical business rules are mirrored at the DB level; for example, orphaned order records are prevented by requiring valid CustomerID associations.
- **Compliance & Retention Policy:** Immutable audit trails shall be preserved for a minimum duration of 24 months to facilitate regulatory compliance and operational dispute resolution.

3.5. SOLID & OOP Architectural Constraints

All source code must strictly enforce the following five SOLID principles. Violations detected during code review constitute a defect of severity MAJOR.

3.5.1 Single Responsibility Principle (SRP)

Layer	Allowed Responsibilities	Prohibited Responsibilities
Model Layer	Data fields, business invariants (non-negative qty/price), encapsulation accessors	Console output, SQL execution, service orchestration
Repository Layer	SQL execution, ResultSet mapping, JDBC connection management	Business rule computation, UI formatting
Service Layer	Core algorithmic business logic, state transitions, financial computations	CLI I/O, direct SQL, presentation formatting
CLI Layer	Text prompt loops, user input parsing, result printing	Business logic computation, DB calls, total calculation

3.5.2 Open/Closed Principle (OCP)

The Promotion processing engine and the Analytics reporting matrix must be dynamically extensible without modification of existing execution logic.

- Adding a new promotional discount type requires only: creating a new class implementing the Promotion interface.
- Adding a new analytics report requires only: creating a new class implementing the AnalyticsReport interface.
- The PromotionEngine and ReportRunner execute against the interface contract; they must not contain switch/if-else trees that enumerate concrete types.

3.5.3 Liskov Substitution Principle (LSP)

The abstract class Party has two concrete subclasses: Seller and Customer. Any service that processes a Party reference must operate correctly regardless of which subclass is passed.

- No service method may use instanceof checks or explicit type casts to differentiate Seller vs Customer processing flow.
- Behavioral contracts defined by Party (e.g., getDisplayName(), isActive()) must be honored by all subclasses.

3.5.4 Interface Segregation Principle (ISP)

Monolithic repository interfaces are prohibited. Each repository interface must expose only the operations relevant to its aggregate root.

Interface	Required Methods (minimum)
ProductRepository	save, findBySku, findBySellerId, updateStock, deactivate, findAllActive

Interface	Required Methods (minimum)
OrderRepository	save, findById, updateStatus, findByCustomerId, findByStatus
ShipmentRepository	save, findById, updateStatus, findByTrackingNumber
PartyRepository	save, findById, findByEmail, findAllSellers, findAllCustomers
AuditRepository	save, findById, findByActor, findInTimeRange

3.5.5 Dependency Inversion Principle (DIP)

High-level modules (Services, CLI) must never depend on concrete repository or service implementations.

- All dependencies must be declared against interfaces, not concrete classes.
- Concrete instances are injected exclusively through constructors (manual dependency injection).
- STRICT: The new keyword is prohibited for instantiating any Repository or Service class inside business logic. Object graph construction occurs only in the application's composition root (main method / factory).

3.6 Software System Attributes

ID	Category	Requirement	Verification Method
NFR-01	[Availability]	All state transitions and inventory mutations must be executed within native JDBC transactions (commit/rollback). No partial writes shall be permitted.	Kill process mid-transaction; verify DB consistency upon restart.
NFR-02	[Correctness]	Financial calculations (subtotal, discount clamping, grand total) must produce results with a precision of 2 decimal places for all test cases.	JUnit unit tests covering edge cases (zero discount, max clamp).

NFR-03	[Maintainability]	Adding a new Promotion or Report type must require zero modifications to existing core service classes (OCP compliance).	Code review; zero diffs in core engine classes when adding a new type.
NFR-04	[Reliability]	Stock rollback during order cancellation must restore the exact quantities originally deducted.	Create order, allocate, cancel, then verify <code>product.availableQty</code> matches the original state.
NFR-05	[Portability]	Shipping carrier list, fees, and free-shipping thresholds must be configurable via external JSON without requiring code recompilation.	Modify JSON, restart, and verify that updated parameters take immediate effect.
NFR-06	[Security]	Every state-changing operation must produce a persisted <code>AuditEvent</code> containing the actor, action, timestamp, and context details.	Query the <code>audit_events</code> table after each operation in integration tests to ensure log entry existence.
NFR-07	[Maintainability]	All JDBC Connection, PreparedStatement, and ResultSet objects must be managed using <code>try-with-resources</code> to ensure zero connection leaks.	Code review checklist; static analysis using SonarLint.

4. Supporting Information

4.1 Table of Contents

[Insert your automatically generated Table of Contents here]

4.2 Index

API 6

Audit Trail 9

JDBC 2, 9

SOLID 9

State Machine 2, 7

4.3 Appendixes

Note: The contents of this section are provided for background information and reference purposes only. Unless explicitly cited within Section 3, these materials are not considered part of the mandatory binding requirements.

Appendix A: Database Schema (DDL)

[Database Schema]

All tables must be created in a dedicated schema (e.g., marketplace_db). The following DDL definitions represent the required baseline schema. InnoDB engine is mandatory to enable foreign key enforcement.

5.1 Table Definitions

parties

Column	MySQL Type	Nullable	Notes
id	VARCHAR(36)	NOT NULL	PRIMARY KEY; UUID string
party_type	ENUM('SELLER','CUSTOMER')	NOT NULL	Discriminator column
name	VARCHAR(200)	NOT NULL	—
email	VARCHAR(320)	NOT NULL	UNIQUE constraint

products

Column	MySQL Type	Nullable	Notes
sku	VARCHAR(50)	NOT NULL	Composite PK (sku, seller_id)
seller_id	VARCHAR(36)	NOT NULL	Composite PK; FK → parties(id)
title	VARCHAR(300)	NOT NULL	—
available_qty	INT UNSIGNED	NOT NULL	CHECK (available_qty >= 0)

Column	MySQL Type	Nullable	Notes
price	DECIMAL(12,2)	NOT NULL	CHECK (price >= 0)
active	TINYINT(1)	NOT NULL	DEFAULT 1

orders

Column	MySQL Type	Nullable	Notes
id	BIGINT	NOT NULL	PRIMARY KEY AUTO_INCREMENT
customer_id	VARCHAR(36)	NOT NULL	FK → parties(id)
status	ENUM('CREATED','AL LOCATED','PACKED',' SHIPPED','CANCELL ED','REFUNDED')	NOT NULL	Default CREATED
subtotal	DECIMAL(14,2)	NOT NULL	—
discount	DECIMAL(14,2)	NOT NULL	DEFAULT 0.00
shipping_fee	DECIMAL(10,2)	NOT NULL	DEFAULT 0.00
grand_total	DECIMAL(14,2)	NOT NULL	—
created_at	BIGINT	NOT NULL	Unix ms timestamp

order_lines

Column	MySQL Type	Nullable	Notes
id	BIGINT	NOT NULL	PRIMARY KEY AUTO_INCREMENT
order_id	BIGINT	NOT NULL	FK → orders(id) ON DELETE CASCADE
sku	VARCHAR(50)	NOT NULL	Snapshot at order time
seller_id	VARCHAR(36)	NOT NULL	Snapshot at order time
qty	INT UNSIGNED	NOT NULL	Requested quantity; CHECK (qty >= 1)
unit_price	DECIMAL(12,2)	NOT NULL	Price snapshot at order creation
line_total	DECIMAL(14,2)	NOT NULL	qty × unit_price
allocation_status	ENUM('FULLY_ALLOC ATED','PARTIALLY_AL LOCATED','UNALLOC ATED')	NOT NULL	Default UNALLOCATED
allocated_qty	INT UNSIGNED	NOT NULL	Actual qty reserved; DEFAULT 0

shipments

Column	MySQL Type	Nullable	Notes
tracking_number	BIGINT	NOT NULL	PRIMARY KEY; monotonic sequence
carrier_code	VARCHAR(20)	NOT NULL	From JSON config; e.g., DHL
order_id	BIGINT	NOT NULL	FK → orders(id); UNIQUE (one shipment per order)
status	ENUM('CREATED','LABEL_PRINTED','IN_TRANSIT','DELIVERED','LOST')	NOT NULL	Default CREATED

audit_events

Column	MySQL Type	Nullable	Notes
id	BIGINT	NOT NULL	PRIMARY KEY AUTO_INCREMENT
order_id	BIGINT	NULL	FK → orders(id); NULL for non-order events
actor	VARCHAR(100)	NOT NULL	Actor identifier
action	VARCHAR(60)	NOT NULL	Event action code
timestamp_millis	BIGINT	NOT NULL	System.currentTimeMillis()
details	TEXT	NULL	Free-form context; required for STOCK_ADJUSTED

]

Appendix B: Integration Payload Examples

Example: Order Placement Request (POST /orders)

```
{  
  "customerId": "CUST-123",  
  "orderLines": [  
    { "sku": "PROD-001", "qty": 2 }  
  ]  
}
```

Appendix C: Project Background

This system was developed to solve the complexity of manual order tracking in multi-vendor marketplaces, providing an automated, auditable, and high-performance backend solution.

Document Status: *This SRS is a living document. We value and encourage technical review and constructive feedback from project reviewers to continuously enhance the system's architecture and requirement clarity*